

REPRODUCIBILITY GUIDE

U.S. Electricity and Water Stress CLI

From official datasets to the two final Python files

What this guide lets a beginner do

Download the three project data files, install the required libraries, train the same four pipelines, and run the final state-and-year stress program.

1. What is being reproduced

Pipeline	Prediction	Final model
SAIDI	Annual outage minutes/customer	Two-stage CatBoost
SAIFI	Annual interruptions/customer	Log-CatBoost
Drought	Drought severity, 0-100	ANOVA + Ridge
Compliance	Violation burden per 100,000	ANN (64, 32)

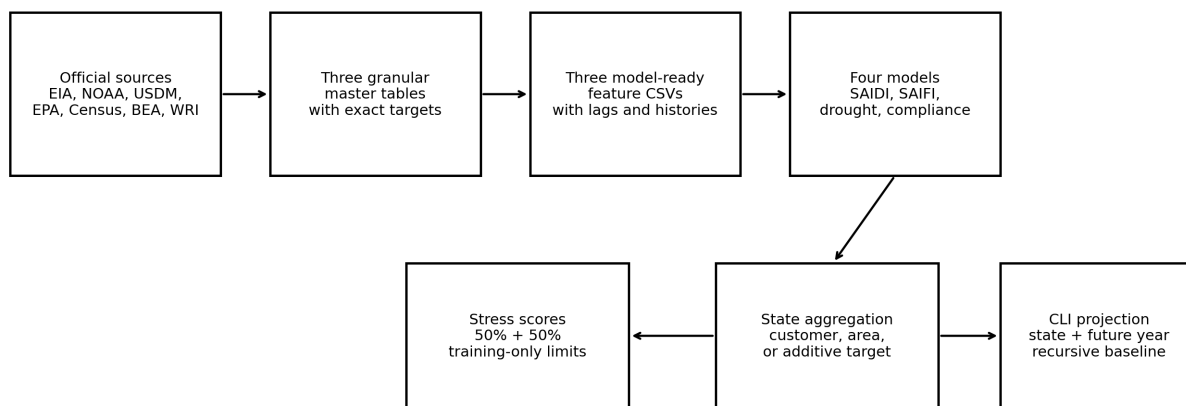
Electricity stress = 50% SAIDI stress + 50% SAIFI stress

Water stress = 50% drought stress + 50% compliance stress

Recommended reproduction route

For an exact and beginner-friendly reproduction, download the frozen model-ready CSVs from GitHub and run the final scripts. Rebuilding from changing raw government downloads requires the project's separate data-building scripts and matching source snapshots.

Reproducible project pipeline



Important: the final training script begins at the model-ready feature CSV stage.

2. Initial official datasets

Each source has a specific role. The tables were combined only after reducing every context source to the correct join level.

Official source	Used for
EIA Form EIA-861	Utility SAIDI/SAIFI targets, customers, ownership and reporting
Other EIA electricity data	Sales, prices, capacity, generation and demand context
NOAA Storm Events	Storm counts, duration and damage
EAGLE-I (ORNL/DOE)	Recorded outage episode context
NREL/OEDI demand	Modeled county electricity demand
U.S. Drought Monitor	Weekly county D0-D4 drought percentages
NOAA county climate	Precipitation, temperature and cooling degree days
Census SAIPE and BEA	Population, poverty, income and GDP
EPA SDWIS	Public water systems and health-based violations
WRI Aqueduct	Static structural water-risk context

Why three data tables create four models

SAIDI and SAIFI share one electricity utility-state-year table because both targets come from the same filing. Drought is weekly county data. Compliance is annual public-water-system data. Joining weekly drought rows directly to annual compliance rows would repeat targets and create false extra observations.

3. How the datasets were combined

Table	One row means	Unique key
Electricity	One utility in one state and year	utility_number + state_fips + year
Drought	One county on one USDM week	county_fips + map_date
Compliance	One public water system in one year	pwsid + year

Electricity

1. Use EIA-861 reliability filings as the target table.
2. Keep SAIDI and SAIFI with major-event days as the targets.
3. Aggregate county context to state-year before joining.
4. Join state context on state_fips + year.
5. Exclude same-year state SAIDI/SAIFI summaries to prevent target leakage.

Drought

```
target_drought_severity_0_100 = (D0 + D1 + D2 + D3 + D4) / 5
```

Join NOAA climate on county_fips + year + month, county socioeconomic data on county_fips + year, and area-weighted Aqueduct context on state_fips.

Compliance

```
row contribution = health-based violation count / state population x 100,000
```

Summing all row contributions within a state-year reconstructs the state target. Served-county lists are summarized before joining so one system is never multiplied into several rows.

4. Cleaning rules

Apply these rules after every download and before any feature engineering.

1. Read ID columns as text so codes such as 01 keep their leading zero.
2. Standardize state FIPS to two digits and county FIPS to five digits.
3. Reduce each context table to one row per intended join key.
4. Use a left join so missing context does not delete target rows.
5. Validate every merge as many_to_one or one_to_one.
6. Check that the base row count is unchanged after a context join.
7. Do not turn every missing value into zero; add availability flags where useful.
8. Sort by entity and time before making lags.
9. Shift before rolling so the current target cannot predict itself.
10. During model evaluation, fit imputation, scaling and feature selection on training rows only.

Safe merge pattern

```
base["state_fips"] = base["state_fips"].astype(str).str.zfill(2)
context["state_fips"] = context["state_fips"].astype(str).str.zfill(2)

context = context.drop_duplicates(["state_fips", "year"])
rows_before = len(base)

combined = base.merge(
    context,
    on=["state_fips", "year"],
    how="left",
    validate="many_to_one",
)

assert len(combined) == rows_before
```

Safe previous-year feature

```
data = data.sort_values(["group_id", "year"])
data["previous_value"] = data.groupby("group_id")["value"].shift(1)
data["previous_year"] = data.groupby("group_id")["year"].shift(1)

data.loc[
    data["previous_year"] != data["year"] - 1,
    "previous_value",
] = None
```

Safe rolling feature

```
data["previous_4_mean"] = (
    data.groupby("county_fips")["target"]
    .transform(lambda values: values.shift(1).rolling(4).mean())
)
```

Why shift(1) matters

Without the shift, the current target can enter its own feature. That would make evaluation look unrealistically good.

5. Model-ready files and features

File	Training record
electricity_features_final_sequence_2013_2024.csv	12 previous-year monthly rows for one annual utility-state target
water_drought_features_county_week_2010_2024.csv	One county-week
water_compliance_features_public_water_system_year_2010_2024.csv	One system-year or labelled residual row

Electricity features

- Previous-year and previous-three-year SAIDI/SAIFI history.
- Monthly sales, customers and modeled electricity demand.
- Storm counts, duration, damage and EAGLE-I outage episode measures.
- Precipitation, temperature, cooling degree days and load factor.
- Capacity, generation, sales ratios and source-availability flags.
- State, utility ownership and reporting categories.

The script confirms exactly 12 months for every sequence, pivots them into one annual row, and treats utility_number, state_fips, ownership and reporting_standard as CatBoost categories.

Drought features

- Recent drought lags and rolling histories.
- Precipitation and temperature history.
- Calendar and seasonal variables.
- County population, poverty, income and GDP.
- Structural Aqueduct risk and data-availability flags.

The final pipeline fills missing predictors with medians, standardizes them, selects the 20 strongest ANOVA F-test features and trains Ridge regression with county-area weights.

Compliance features

- Previous system violation history and system inventory context.
- State population, poverty, income and other socioeconomic context.
- Climate, drought, water-use, plumbing and infrastructure context.
- State and master-record-type dummy columns plus missing-data flags.

The target is transformed with log1p. The selected neural network has hidden layers of 64 and 32 nodes, ReLU activation, Adam optimization and early stopping.

Do not use target columns as features

Targets, target decompositions, IDs, sequence IDs, years and weighting columns are excluded from predictors unless they are valid lagged historical values.

6. Beginner setup: download and train

Required folder layout

```
project/  
|-- data/           created by the downloader  
|-- saved_models/   created by training  
|-- download_master_data.py  
|-- train_models_once.py  
|-- stress_score_cli.py  
|-- requirements.txt  
`-- Beginner_Reproducibility_Guide.pdf
```

Step 1: install Python 3.13

On Windows, select Add Python to PATH during installation. Confirm the installation:

```
py -3.13 --version
```

Step 2: clone the repository

```
git clone https://github.com/amessbee/rise26project.git  
cd rise26project.
```

Step 3: create a private Python environment

```
py -3.13 -m venv .venv  
.\venv\Scripts\Activate.ps1
```

The terminal normally begins with (.venv) when the environment is active.

Step 4: install the libraries

```
python -m pip install --upgrade pip  
python -m pip install -r requirements.txt
```

Step 5: download the three project CSVs

```
python csv_downloader.py
```

The script downloads three CSV

```
All project data files are ready.  
Next command: python train_models_once.py
```

Step 6: train and save the models

```
python train_models_once.py
```

Training can take a long time and requires substantial memory because the water tables contain millions of rows.

```
ALL FOUR PIPELINES HAVE BEEN TRAINED AND SAVED
```

The script creates saved_models/. Running it again does not train again. To deliberately replace all saved models:

```
python train_models_once.py --force
```

Step 7: run the final project

```
python stress_score_cli.py
```

Enter a state such as Texas, texas, TX or tx, then a four-digit year after 2024.

7. What the two final Python files do

train_models_once.py

- 1. Checks that all three model-ready CSVs exist.
- 2. Turns every 12-month electricity sequence into one annual row.
- 3. Uses chronological training and validation years to tune CatBoost iterations.
- 4. Trains the selected SAIDI, SAIFI, drought and compliance pipelines.
- 5. Aggregates results to state-year using customers, county area or additive compliance contributions.
- 6. Saves models, histories and metadata inside saved_models/.

Pipeline	Simple explanation
SAIDI	Classify high-event years, then blend normal and high CatBoost predictions
SAIFI	Predict log1p(SAIFI) with CatBoost and convert back
Drought	Select 20 features with ANOVA, then fit Ridge regression
Compliance	Fit a 64 -> 32 node ANN to the log-transformed target

stress_score_cli.py

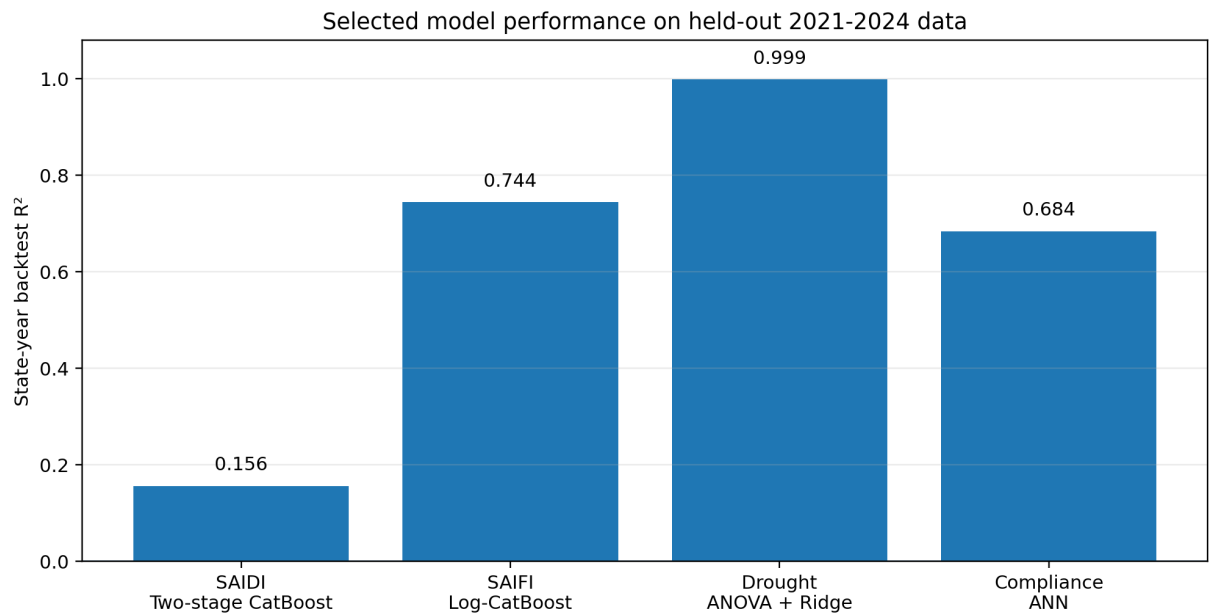
The CLI does not retrain the models. It loads state_model_history.csv and project_metadata.json. It begins with the latest observed state value, measures the recent model-estimated direction, predicts one year at a time, halves the trend after each step and gradually pulls the result toward the state's historical conditions.

8. Chronological split and selected results

Stage	Years	Purpose
Training	through 2018	Learn model parameters
Validation	2019-2020	Choose settings and architectures
Backtest	2021-2024	Final held-out evaluation

Selected state-year backtest results

Pipeline	MAE	RMSE	R ²
SAIDI - two-stage CatBoost	205.092	452.906	0.156
SAIFI - Log-CatBoost	0.236	0.318	0.744
Drought - ANOVA + Ridge	0.466	0.585	0.999
Compliance - ANN	2.548	4.341	0.684



Stress-score calculation

SAIDI, SAIFI and compliance are converted to 0-100 using 5th and 95th percentile boundaries calculated from training years only. Values beyond the boundaries are clipped to 0 or 100. Drought is already 0-100.

9. Final reproduction checklist

- The three exact CSV filenames exist inside data/.
- Training creates saved_models
- The CLI opens without a missing-file error.
- Both state names and abbreviations are accepted.
- The selected year is later than 2024.
- The output shows SAIDI, SAIFI, drought, compliance and both final stress scores.

Common problems

Problem	Fix
CSV not found	Run python download_master_data.py and keep the exact filenames
Memory error	Close other programs or train on the stronger computer/server
Library not found	Activate .venv and reinstall requirements.txt
Models already trained	This is normal; use --force only when replacing them
GitHub Release not found	Check the repository name

What must be included in the GitHub repository

- The two final scripts: train_models_once.py and stress_score_cli.py.
- download_master_data.py and requirements.txt.
- A GitHub Release tagged data-v1 with the three ZIP files.
- This PDF and the editable Markdown version.
- For full raw-source rebuilding: the data-building scripts and the exact official source snapshots or download instructions.

One-command summary

```
py -3.13 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python download_master_data.py
python train_models_once.py
python stress_score_cli.py
```

Reproduction complete

A beginner has reproduced the final project when the data download succeeds, the four saved pipelines are created, and the CLI returns both electricity and water stress scores for a selected state and future year.